Comp 533 - Assignment 6: Object Replication Using Non-Blocking Byte Communication

Date Assigned: Thursday, April 10, 2025

Early Completion Date: Friday, April 25, 2025, 11:55 pm (+5% extra credit)

Completion Date: Monday, April 28, 2025, 11:55 pm

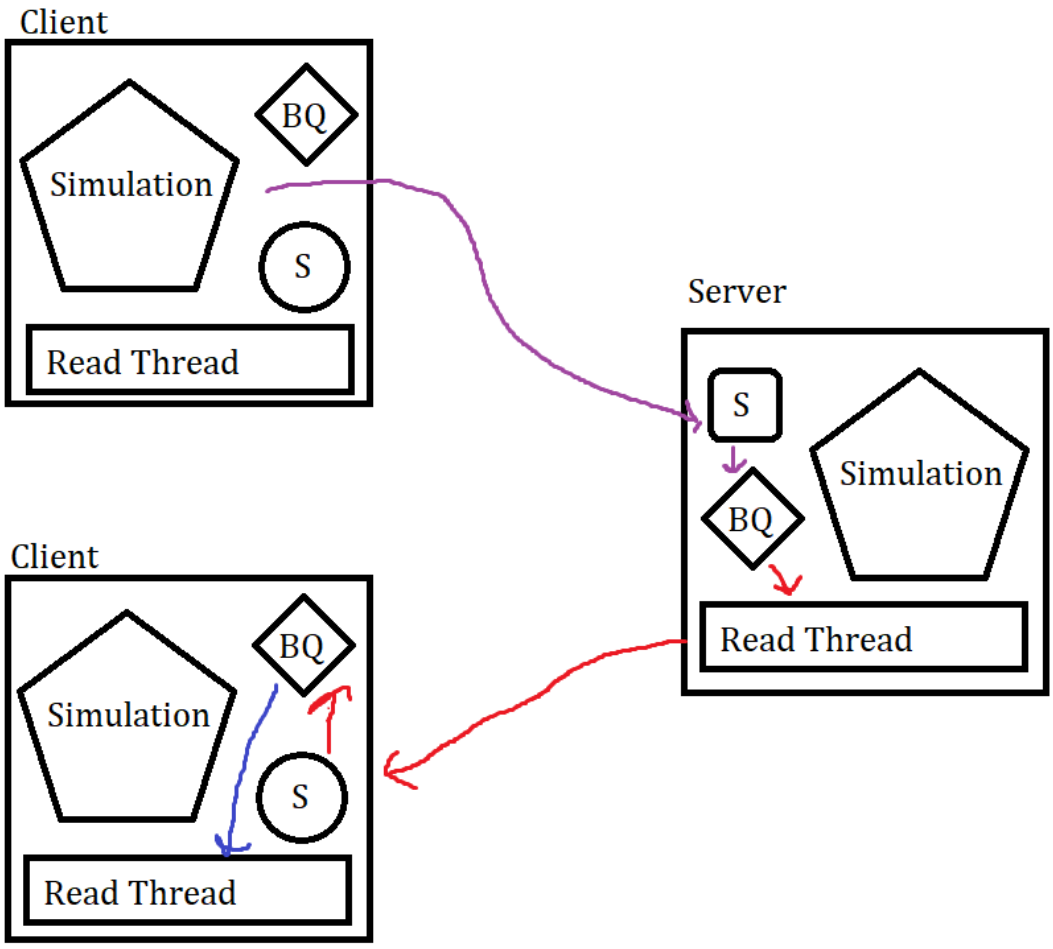
No Late Submission!

The goal of the assignment is to learn and use scalable non-blocking communication mechanisms. The assignment will expose you to the listener-based NIOManager for using NIO. You will add to your previous assignment another way of exchanging messages among the clients and processes based on using Java NIO via NIOManager.

The assignment will involve creating your own thread that interacts with the selecting thread provided by the NIO manager. The classic producer-consumer relationship connects the two threads. Hopefully, your experience with A2 will help you with this task.

Here is a summary of the recommended steps:

1. Understand the distributed printer application described in the lecture material, which will involve understanding the material on which it builds (Bytebuffer, ServerSocketChannel, SocketChannel, …) You may not understand all the subtleties of NIOManager at this point.
2. Extend the example code to make the server echo the received string to each client and the client to receive and print this value. You will need to play around with interestops.
3. Modify the example code to echo each received value to each connected client except for the client that initially sent it. You will now need to save the set of connected socket channels in global variables.
4. Integrate the example code with A5.



5.

The main challenges in this assignment are implementing the producer-consumer relationship and determining the state machine describing the changes in channel interestops needed for communicating changes.

You will *add* the NIO Mechanism to your A5 solution rather than create a standalone NIO solution. This means the clients and the server can talk to each other using RMI, GIPC or NIO. The choice of the IPC mechanism to be used will be decided by the meta object IPC mechanism, you implemented in the previous assignment.

As in A5, the IPC mechanism meta object will be replicated using only RMI.

Scaffolding Code and Learning Material

We will continue to build on the solution from the previous assignment, now adding NIO as a means of communicating between the different client processes.

You should use the higher-level abstraction, called the NIOManager, in your assignment. Its use is described in the print example.

As in previous assignment, the main arguments do not need to be specified because of the provided defaults (this may change with extra credit opportunities in future assignments).

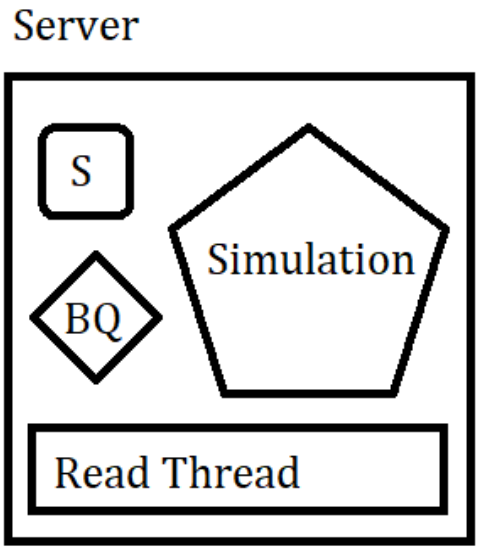
Assignment Visualization

Use this visualization to give you an idea of what's required for the assignment.

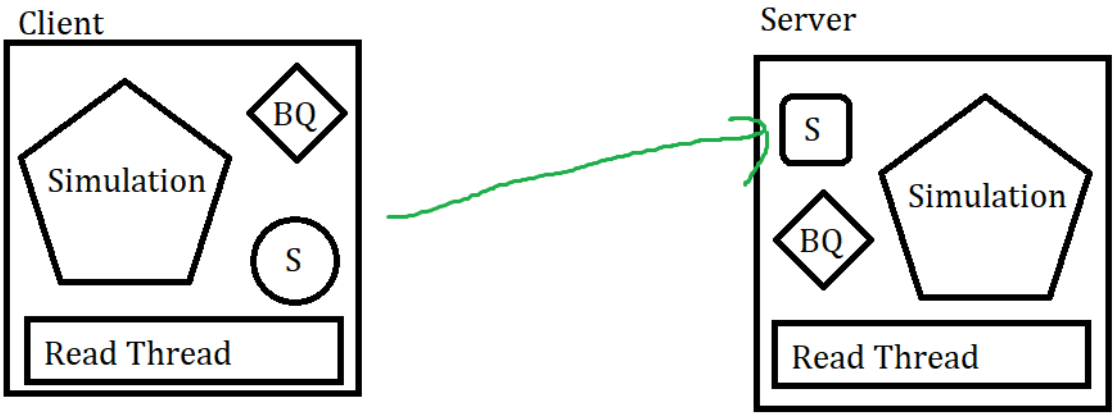
As we saw in the lecture, the selection thread created by NIOManager should not block after processing an incoming message. In fact, it should take very little time in loop iterations. One way to ensure that is to have the read callbacks invoked by enqueueing their data in a queue that is dequeued by a read thread. This queue is blocking in that the read thread blocks if it is empty. However, the read callbacks, executed by the select thread, do not block if the queue is full, and can give an error message if this happens.

This is the architecture you need to implement. As the selection thread is created in both the client and server, this needs to be done in both the server and client programs.

When the server initializes, it opens a server socket channel and creates its Read Thread. The Read Thread invokes blocking dequeue operations on the server blocking queue.



The client creates/opens its own socket channel and creates its own Read Thread which also invokes blocking dequeues on its queue. The client then connects with the server and the server stores the client socket to implement broadcasts.



When a client reads an input, it sends a message containing the input to the server using the NIOManager write method. It also processes the input, of course, by delivering the input to its simulation. The message received by the server NIOManager makes it to the server Read Thread through the server read listener. The Server Read Thread then checks the client socket channel list and forwards the message from a client's socket channels to all other clients' NIOManagers which put it in their own blocking queues. The message then makes it to each client's Read Thread.

From here each client Read Thread finishes the update to the client and blocks again on a dequeue operation on the blocking queue. Similarly, after broadcasting a message taken from its blocking queue, the server Read Thread executes a blocking dequeue on its queue.

This section has outlined the new conceptual challenge in this assignment. The rest of the sections describe all of the tasks.

Select Interest Operations

You need to determine the steady state interestops of the server and client end points of the channel – whether they should include SelectionKey.READ_OP or not. The steady state interestops are the interestops when the channel is not being written or connected. The accept and connect operations take the value of this argument, which is expected to be either 0 or SelectionKey.READ_OP.

Separate Thread for Processing Incoming Messages

In general, we do not want to do application-specific processing in the selecting thread, as during this processing, the thread cannot process incoming messages and could possibly block, creating a deadlock. Therefore, as mentioned above, to create a more realistic application and gain practice with threads, ensure that in both the server and client, a separate thread processes read messages. *For automatic grading, use the setName() Thread method to give the thread the name defined by the following constant*:

```
public static final String READ_THREAD_NAME = "Read Thread";
```

The read listener, executed by the selecting thread, will enqueue information about the read messages into a blocking queue or bounded buffer, and your read thread will

dequeue this information and process it. This means a message received by the channel (through the kernel) is first transferred to the selecting thread through a read call. The read call copies the channel data into a read buffer passed to the read call. This buffer is then passed as an argument to your application-specific read listener, which enqueues its contents in the bounded buffer. The read thread dequeues items from this queue.

The NIO Manager library uses a single byte buffer for all reads from the kernel buffers. This means the library buffer changes after each read. This does not cause problems if each read is processed by its listening clients before the next read is delivered to the listeners. As your listener will be queuing read data to be consumed by the Read Thread, it is important to store copies of the read buffer passed to your listener. Otherwise, each read by the selecting thread will influence the data references in all queued items.

Use the deepDuplicate() method in the class MiscAssignmentUtils in assignments.util in the library jar to make the copy. Do not convert the ByteBuffer to a String as that makes your server a String relayer rather than a more general bytebuffer relayer and also involves an extra copy (ByteBuffer-->String-->ByteBuffer).

Use a Java ArrayBlockingQueue implementation of a bounded buffer/blocking queue and define an appropriate data type (class) to define the enqueued items. The reading thread should use the take() operation to consume items from the buffer, which will block the thread until it is non-empty. However, do not use the put() operation to add elements to it, as that will block the selecting thread until there is space. We never want the selecting thread to block. So the read listener should use the non-blocking add() method if there is remaining space, and print an error message if there is not. Choose a large number for the size of the array, which will depend on the timing experiments mentioned below. Make sure your buffer does not overflow during the experiment.

Threads for User-Interaction Processing

While the read thread is the only thread you will create, there are other threads making your programs work. NIOManager creates the selecting thread, which produces items consumed by the read thread. Java creates the main thread, which processes input commands in a loop. In a client, the UI code uses the AWT toolkit, which creates an AWT thread the first time the simulation frame is made visible.

Tags and Main Arguments

As in A4 and A5, To help the auto grader find the server and client, you should tag the classes by prefixing the class declaration with tagging annotations in the manner shown below:

```
import util.annotations.Tags;
import util.tags.DistributedTags;
@Tags({DistributedTags.RMI, DistributedTags.GIPC, DistributedTags.NIO, DistributedTags.SERVER})
public class <Server Name> …
----------------------------------------
import util.annotations.Tags;
```

```
import util.tags.DistributedTags;
@Tags({DistributedTags.RMI, DistributedTags.GIPC, DistributedTags.NIO, DistributedTags.CLIENT})
public class <Client Name> …
```

Like the example and the previous assignments, both the server and client must use the main args library to determine host and/or port specifiers.

These are the expected tags for source checks:

Unchanged:

@DistributedTags.SERVER_REMOTE_INTERFACE+@DistributedTags.RMI+@DistributedTags.GIPC,

@DistributedTags.REGISTRY+@DistributedTags.RMI,

@DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.RMI,

@DistributedTags.CLIENT_REMOTE_INTERFACE+@DistributedTags.GIPC,

@DistributedTags.CLIENT_REMOTE_OBJECT+@DistributedTags.RMI+@DistributedTags.GIPC,

@DistributedTags.SERVER_REMOTE_OBJECT+@DistributedTags.RMI+@DistributedTags.GIPC,

Updated:

The server configure must now also create NIO server channel end points through the NIO Manager.

@DistributedTags.SERVER_CONFIGURER+@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO,

A single client main must support all three IPC mechanisms:

@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO+@DistributedTags.CLIENT,

The previous out coupler now calls an external method that decides how to send observed change in the simulation:

@DistributedTags.CLIENT_OUT_COUPLER+@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO,

The client configure must now also connect to NIO server channel end points through the NIO Manager.

@DistributedTags.CLIENT_CONFIGURER+@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO,

A single server main must support all three IPC mechanisms:

@DistributedTags.RMI+@DistributedTags.GIPC+@DistributedTags.NIO+@DistributedTags.SERVER,

New:

The same interface can be used for the client and server runnables created for server and read threads . This is, of course, an extension of the Java Runnable interface.

@DistributedTags.NIO+@DistributedTags.READ_THREAD_INTERFACE

This is the class of the server runnable implementing the above interface:

@DistributedTags.NIO+@DistributedTags.SERVER_READ_THREAD

It implements the interface above.

This is the class of the client runnable implementing the interface above

@DistributedTags.NIO+@DistributedTags.CLIENT_READ_THREAD

It implements the interface above.

Tracing

Your server and client must now also enable NIO traces:

NIOTraceUtility.setTracing();

FactoryTraceUtility.setTracing();

BeanTraceUtility.*setTracing*();

RMITraceUtility.*setTracing*();

ConsensusTraceUtility.*setTracing();*

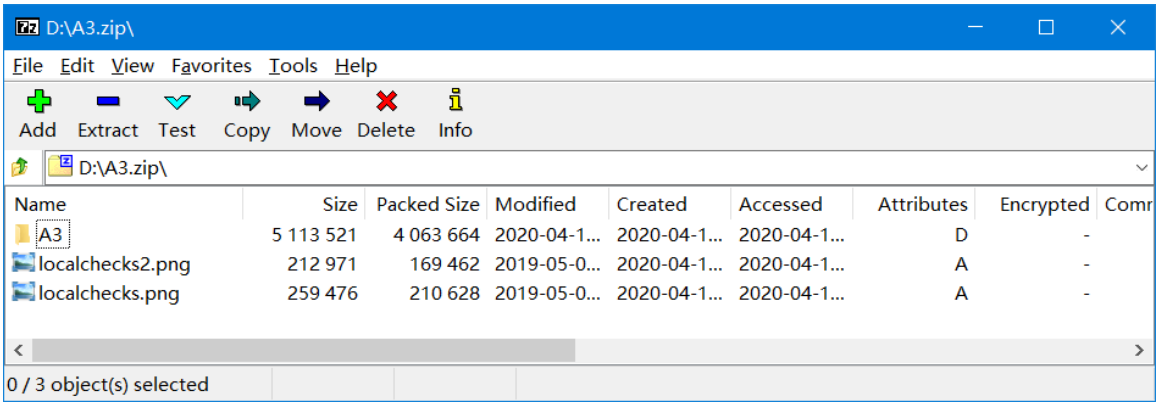
ThreadDelayed.*enablePrint();*

GIPCRPCTraceUtility.setTracing();

.

Submission Instructions

Same as those for A5



Performance Experiments (Not graded)

By extending the simulation with three different IPC mechanisms you can evaluate the effect on performance of using each IPC mechanism to communicate commands. There are three conditions:

local processing only, when commands are not communicated,

RMI use, when commands are executed using synchronous RMI RPC,

GIPC use, when commands are executed using asynchronous GIPC RPC,

NIO use, when commands are executed using asynchronous NIO IPC.

You need to execute a series of commands to increase the reliability of the results.

To do so, the scaffolding code supports two interactive commands and corresponding callbacks in the client and server.

l(ocalProcessingOnly)

You need to compare the performance overhead of your collaborating implementation. You can do this by running a client under two conditions:

(1): Local Processing Only: A command undergoes only local processing. In this mode, your client behaves like the original simulation you extended – it does not transmit any message in response to a command. It can react to incoming commands, or not - your decision.

(2): Regular Processing: Each command is processed not only locally but also broadcast using whatever IPC mechanism has been set by the **i** command.

How to react to this command is illustrated in the coupling template. This command makes no sense in the coupling template, but to illustrate how you should implement it, the template includes an implementation that dynamically removes (add) the coupling between the two command models when local processing only is true/false:

```
@Override
public void localProcessingOnly(boolean newValue) {
        super.localProcessingOnly(newValue);
```

```
            if (isLocalProcessingOnly()) {
commandProcessor1.removePropertyChangeListener(simulation2Coupler);
commandProcessor2.removePropertyChangeListener(simulation1Coupler);
            } else {
commandProcessor1.addPropertyChangeListener(simulation2Coupler);
commandProcessor2.addPropertyChangeListener(simulation1Coupler);
            }
      }
```

Here is an interactive invocation of the code above:

```
l true
public default void
assignments.util.inputParameters.SimulationParametersListener.localProcessingO
nly(boolean)[true]
I***@1517598325259700 {main}(RemovedPropertyChangeListener)
EvtSrc(Your_Halloween_Processor_Class_Name)
coupledsims.ASimulationCoupler@4250b0c1
I***@1517598325621100 {main}(RemovedPropertyChangeListener)
EvtSrc(Your_Halloween_Processor_Class_Name)
coupledsims.ASimulationCoupler@7b5e5fa
```

You will thus have to change the implementation of this method in your client and server implementations. The server version will do nothing while the client version will behave as described above.

e(xperimentalInput)

As mentioned above, an important issue in distributed computing, of course, is performance. The absolute performance is not as important as the relative performance of different design decisions. Such performance requires repeating a set of actions under different alternatives. This command is expected to trigger such a repetition. It calls the following abstract parameter method in the template:

```
@Override
      public void experimentInput() {
            long aStartTime = System.currentTimeMillis();
            PerformanceExperimentStarted.newCase(this, aStartTime,
NUM_EXPERIMENT_COMMANDS);
            boolean anOldValue = isTrace();
```

```
            this.trace(false);
            for (int i = 0; i < NUM_EXPERIMENT_COMMANDS; i++) {
                  commandProcessor1.setInputString(EXPERIMENT_COMMAND_1);
                  commandProcessor1.setInputString(EXPERIMENT_COMMAND_2);
            }
            trace(anOldValue);
            long anEndTime = System.currentTimeMillis();
            PerformanceExperimentEnded.newCase(this, aStartTime, anEndTime,
anEndTime - aStartTime, NUM_EXPERIMENT_COMMANDS);


      }
```

```
Enter (prefixes of) print, quit or a method and its parameters separated by
commas:
e
public default void
assignments.util.inputParameters.SimulationParametersListener.experimentInput(
)[]
I***@1517562909275000 {main}(PerformanceExperimentStarted)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)  1582389387721:500
I***Tracer: showInfo = false
I***Tracer: showInfo = true
I***@1517571674764800 {main}(PerformanceExperimentEnded)
EvtSrc(AStandAloneTwoCoupledHalloweenSimulations)
1582389387721-1582389396487=8766:500
```

The template implementation executes 500 simulation commands and prints the time required to process these commands. While doing so, it disables tracing, which can not only blow up the console output but also significantly increase the processing time. You can use this implementation directly in your client coupled simulation. If you do not disable traces, you will get a message complaining about too many traces.

Performance experiment

Determine the result of timing of the execution of 500 input commands at a site issuing these commands under the following conditions:

- Local execution only: the simulation executes the commands locally without sending any messages to remote sites.
- Basic Distributed 3-User RMI: Same as above except that commands are broadcast to two other simulations using RMI.
- Basic Distributed 3-User GIPC: Same as above except that GIPC is used for timing.

- Basic Distributed 3-User NIO: Same as above except that NIO is used for timing,

This part will not be graded but you might be asked to post the results a chat Q/A question and asked to explain the results. Even if you are no asked to do so in class, performance comparisons can be asked in the exam.

Possible Coding Issues

1. The client select is blocked on a read.
    a. Look at the NIO Manager section on how to fix this.
    b. Search for "interest op".
2. Does a new thread process each dequeued item?
    a. if we made a new thread each time, then we would not, in fact, have to queue the requests. The selecting thread can simply execute the thread rather and not need the intermediate thread. However, this is inefficient.
3. deepDuplicate is not needed if the received bytebuffer is converted to a String
    a. Converting the received message to a string makes the server a string relayer rather than a more general bytebuffer relayer and also involves an extra copy (bytebuffer-->string-->bytebuffer).
4. Should the reading thread in the server or client call listeners?
    a. No, the listeners are called by the selecting thread, and they add items to the bounded buffer consumed by the reading thread.
5. Should the timing results determine how long it took to process the commands in the client
    a. No, the performance experiments will end before the client has received the commands. The experiment measures the time taken by the main thread to write to the socket channel.
6. The number of reads is sometimes not equal to the number of messages received but is equal to the number of commands sent.
    a. Yes, the NIO manager handles the non-deterministic scatter-gather.
7. I set the reading thread to have Read Thread but grader says "Didn't find server read thread. Didn't find client read thread"n. Test Name: OneClientThreadAtomic Timeouts are probably too small, see part in A4 on timeouts.
8. If you get the error message, "In Client writing to server, no line found matching regex: .*?\{main\}.*?NotifiedPropertyChangeEvent.*" make sure the SimulationParametersListener is running in your main thread, not running on a new thread that you created

9. Server message is received by only the sending client or none of the clients

   Make sure you have instantiated the connect factory correctly in the client.

10. Server or client seems to stop in the middle of its expected execution script: You may have called a thread's infinite run loop directly instead of through the Thread start method. This in turn implies you did threads the wrong way by subclassing Thread rather than passing a runnable to Thread's constructor.
11. Tests cannot find tags. Make sure you run them from your project and not Comp533LocalChecks or some other project.
12. Tests do not show traces when they fail:

    Make sure Tracer.showInfo(true) is called from RunTests.

13. Another issue that could be causing error messages such as "In Server writing to client 1, no line found matching regex: .*?\{Read Thread\}.*?WriteRequestEnqueued.* is not adding new lines after personal debug print statements. The grader is checking the beginning of each line in the console to match expected traces, so if there is no new line after a print statement, the trace information gets appended to the end of that debug print line and the grader doesn't find it
14. If the client seems to be sending twice, make sure that the halloween command processor isn't registering the sender as a receiver twice.
15. If your write is not going through in the server or client, you might be writing to the socket channel directly instead of going through the NIOManager.
16. If you are getting a mangled string in a receiving client (some boxes and the actual string), you probably did not do a deepDuplicate in the server of the received message.
17. If you get the message about a factory not being sent, you might be calling the looping processCommands() before the factory setting and other code - remember it is always the last action the main thread takes.
18. If setMaxTraces() does not seem to work for you, make sure it is being set in both the client and server.
19. Your RMI shouldn't need to use a server port - just a registry port. That's the port where you'll be looking for the registry, binding the server and looking up the server.Otherwise you may get a message saying RMI cannot start because NIO is already using port 9000.
20. Make sure you set the accept command factory before opening socketChannel. You may otherwise get a message saying the tests were unable to find a trace of SocketChannelBound
21. Setting traces off in one process does not set them off in other processes, you must manually do so, or you can add a feature to broadcast it automatically.
22. Make sure that when your client/server receives a meta state update, the receiving client/server uses the setIPCMechanism() function instead of the ipcMechanism() function of SimulationBean. This can cause a deadlock.
23. Make sure the RMI connection is made first from the client since all broadcasts happen through it.

GPT Summary and Steps

A concise summary of the **key points from Assignment 6: Object Replication Using Non-Blocking Byte Communication** for **Comp 533**:

🎯 Goal

Extend your solution from Assignment 5 to support **Java NIO (Non-blocking I/O)** using **NIOManager**, in addition to existing **RMI** and **GIPC** communication mechanisms. This makes your system support **three IPC mechanisms** with dynamic switching via the ipc_mechanism meta object.

🔧 Main Implementation Tasks

1. **Integrate NIOManager-based communication** between clients and server.
    - Clients send commands to server via **NIO socket channel**.
    - Server **broadcasts** received commands to all other clients (except sender) via their socket channels.
2. **Add Read Threads** (both client and server):
    - Use a **producer-consumer** model with a **bounded blocking queue**.
    - Selecting thread enqueues messages into queue via a **read listener**.
    - A separate **Read Thread** (name = "Read Thread") dequeues and processes messages.
3. **Modify IPC Switching Meta Object**:
    - Keep using RMI to replicate ipc_mechanism.
    - Add "i NIO" command support for switching to the NIO IPC mechanism.

📬 Message Flow with NIO

- Client sends command → NIOManager writes to server → server's selecting thread reads it → server's Read Thread forwards it to other clients → those clients enqueue and process via their Read Thread.

🧱 Core Components

- Use deepDuplicate() to copy byte buffers before enqueuing.
- Use ArrayBlockingQueue (non-blocking add() in listener, take() in Read Thread).
- Avoid converting buffers to strings in server – remain a **bytebuffer relayer**.

🏷️ Required Annotations

Update your classes with the following tags:

Component	Tags
Server Main Class	SERVER, RMI, GIPC, NIO
Client Main Class	CLIENT, RMI, GIPC, NIO
Server Configurer	SERVER_CONFIGURER, RMI, GIPC, NIO
Client Configurer	CLIENT_CONFIGURER, RMI, GIPC, NIO
Out Coupler	CLIENT_OUT_COUPLER, RMI, GIPC, NIO
Read Thread Interface	NIO, READ_THREAD_INTERFACE
Server Read Thread Class	NIO, SERVER_READ_THREAD
Client Read Thread Class	NIO, CLIENT_READ_THREAD

📊 Performance Evaluation (Optional)

Use provided scaffolding for **performance experiments** with commands:

- l true → Local-only processing
- e → Run 500 simulation commands and log execution time

Compare performance across:

- Local-only
- RMI (synchronous)
- GIPC (async, non-blocking)
- NIO (async, non-blocking)

🧪 Required Traces

Enable all tracing utilities:

java

CopyEdit

NIOTraceUtility.setTracing();

FactoryTraceUtility.setTracing();

BeanTraceUtility.setTracing();

RMITraceUtility.setTracing();

ConsensusTraceUtility.setTracing();

ThreadDelayed.enablePrint();

GIPCRPCTraceUtility.setTracing();

Make sure traces like the following appear:

- ReadListenerAdded
- SocketChannelWriteRequested
- WriteRequestEnqueued
- SocketChannelWritten
- SocketChannelFullMessageRead
- SelectCalled, SelectorWokenUp, etc.

⚠️ Common Pitfalls

- Not using deepDuplicate() → corrupt shared buffers
- Blocking the selecting thread (avoid blocking queues in listeners)
- Incorrect thread naming (Read Thread)
- Registering read listeners or starting threads in the wrong order
- Not updating ipc_mechanism correctly across components
- Writing directly to SocketChannel instead of via NIOManager

Would you like a checklist version of this summary or a performance experiment template to use in your code?

4o

You said:

Yes

ChatGPT said:

Here's a **checklist version** of the Assignment 6 summary and a **performance experiment code template** you can plug into your project. This will help ensure you're covering all implementation requirements and have a clear way to run timing tests.

✅ Assignment 6 Implementation Checklist

🛠️ Core Integration

- Extend A5 solution to support NIO as a third IPC mechanism.
- Add NIO socket channel support to both server and client.
- Register NIO read listeners via NIOManager.

📤 Sending & Receiving Messages

- Client uses NIOManager.write() to send command to server.
- Server reads via selecting thread → enqueues into blocking queue.
- Server Read Thread dequeues, forwards to other clients via their NIOManager.
- Clients receive and enqueue → client Read Threads process them.

🔁 IPC Mechanism Meta Object

- Handle i NIO console command in ipcMechanism(IPCMechanism newValue).
- Only **sending side** uses the selected IPC mechanism.
- Meta object still replicated using **RMI only**.

🧵 Threading Setup

- Implement a Read Thread in both client and server:
  - Thread name set using: setName("Read Thread")
  - Consumes from ArrayBlockingQueue via take()
- Listener (executed by selecting thread) uses add() (not put())
- Use deepDuplicate() to copy ByteBuffer before enqueueing

🏷️ Required Tags

- @DistributedTags.SERVER, RMI, GIPC, NIO
- @DistributedTags.CLIENT, RMI, GIPC, NIO
- @DistributedTags.SERVER_CONFIGURER, CLIENT_CONFIGURER
- @DistributedTags.CLIENT_OUT_COUPLER
- @DistributedTags.NIO, READ_THREAD_INTERFACE, SERVER_READ_THREAD, CLIENT_READ_THREAD

📊 Performance Experiment Support

- Implement localProcessingOnly(boolean) in client.
- Implement experimentInput() in client.

🧪 Performance Experiment Code Template

Use this in your **client** class (which extends AnAbstractSimulationParametersBean) to test 500 commands:

java

CopyEdit

@Override

public void experimentInput() {

long startTime = System.currentTimeMillis();

PerformanceExperimentStarted.newCase(this, startTime, NUM_EXPERIMENT_COMMANDS);

```
    boolean oldTracing = isTrace();
    this.trace(false); // Disable trace for speed

    for (int i = 0; i < NUM_EXPERIMENT_COMMANDS; i++) {
        commandProcessor1.setInputString(EXPERIMENT_COMMAND_1);
        commandProcessor1.setInputString(EXPERIMENT_COMMAND_2);
    }

    this.trace(oldTracing); // Restore trace setting
    long endTime = System.currentTimeMillis();

    PerformanceExperimentEnded.newCase(this, startTime, endTime, endTime -
startTime, NUM_EXPERIMENT_COMMANDS);
}
```

And to support local-only mode (no network messages):

java

CopyEdit

```
@Override
public void localProcessingOnly(boolean newValue) {
    super.localProcessingOnly(newValue);

    if (isLocalProcessingOnly()) {
        commandProcessor1.removePropertyChangeListener(simulation2Coupler);
        commandProcessor2.removePropertyChangeListener(simulation1Coupler);
    } else {
        commandProcessor1.addPropertyChangeListener(simulation2Coupler);
        commandProcessor2.addPropertyChangeListener(simulation1Coupler);
    }
}
```

✅ **To Run Experiments:**

Use the following **interactive commands** in the console:

- l true → Enable local-only processing
- l false → Resume distributed communication

- i NIO / i RMI / i GIPC → Select IPC mechanism
- e → Run experiment and log duration

Reference Material (mainly for the curious)

This material is mainly for the curious.

Overview of NIO

You need to look at the Rox tutorial and/or the class material on NIO programming to properly understand NIO. Here is a superficial overview of the material.

Each client creates an individual physical channel connected to a logical channel end-point published by the server. The endpoint is named by hostname, an integer called the port number. The individual channel is created when the client *connects* to the end-point, and the server *accepts* the request. The individual channels are separate – the messages sent on them are not mixed in any system-provided queue, and the server processes each sequence of messages independently. Hence the adjective logical for the end-point. Once an individual channel is established between a client and server, both parties can send (write) and receive (read) byte sequences.

The read and write operations work in terms of ByteBuffer objects. Such an object encapsulates an array of bytes along with pointers that mark positions in the buffer that allow a reader or writer to know where to start reading/writing from and how much is to be read or has been written. These pointers support encapsulation and allow reuse of the same byte sequence for multiple reads and writes. We will refer to these objects as buffers. A written buffer must be "flipped" (using the flip() operation) to make the written contents readable. The wrap method used in the example automatically flips the ByteBuffer after it writes to it.

A buffer written to a channel by a writer may be received in or scattered into multiple buffers read by the reader, which thus needs to gather the scattered written buffer into one buffer. Conversely, multiple buffers sent by a writer may be received or gathered in a single buffer read by the reader, which thus needs to scatter into multiple buffers the single gathered buffers.

NIO can support both blocking and non-blocking I/O – we will use the default non-blocking mode. In non-blocking I/O, threads that invoke channel operations (accept, connect, read, write) do not block. They invoke the operations when these operations are ready to execute. In a non-polling solution, some other thread, however, must block, waiting for ready operations.

NIO provides a select() operation in a selector object for the thread to block until some operation of interest is ready to execute. The blocking semantics depends on the operation. Blocking for (a) read means waiting until 1 or more bytes are received on the specified channel from the other end, (b) write means waiting until there is buffer space in the kernel to send at least one byte on the channel, sending the bytes that can be copied into the kernel buffer, and returning without sending the bytes that could not be copied, (c) accept means waiting until a connect request is received from a remote end, (d) connect means waiting until the connection is accepted by the remote end.

The selector object can be told which channels it should block for by registering channels with it. At registration time, the selector can be also told the set of channel operations it should wait for – called the interest operations (interestops). The selector unblocks when any of these operations is ready to execute. The set of interest operations can change dynamically. For instance, when an application needs to write, it can ask the selector to block for the write, and after the write completes, if incoming data is needed, it can be told to block for a read.

A thread blocked on a select() operation can be unblocked by invoking the wakeup() operation on the selector. This is important when the interest operations change dynamically.

I encourage you to use the Rox tutorial for really understanding NIO. One feature the library does not show is the keyFor() method in a SocketChannel(), which given a SocketChannel and Selector, returns the SelectionKey instance, on which you can execute interestOps() operations. This feature, along with most of the ones mentioned above, is exposed using a higher-level provided by the NIO Manager and is needed only to understand how the NIO manager works – not its use.

Understanding NIO Manager

The Rox tutorial suggests that it is unsafe to simultaneously block for both reads and writes on a channel, or to have multiple threads execute operations to block or change the blocking interest operations. The (GIPC) NIO Manager provides a built-in library to provide a single generic application-independent selecting thread that blocks on a selector object and changes its interest operations. It must, thus, provide a way to interface with this generic thread, which is implemented using the concept of "asynchronous" command objects. Different kinds of such objects are associated with different NIO operations. We did not cover command objects in class, so their discussion is for the curious.

In general, a command object captures a method call to be invoked at some later time by some system with which it is registered such as a thread, an undoer or Java executor service of future objects. It provides an execute() method to make this call, and the registering and invoking system is oblivious to what the call does. After construction, a command object is put in an appropriate registration queue, processed by the selecting thread. The thread performs the initialize() methods at dequeue()/registration time, and the execute() method when the corresponding operation is ready to execute. Thus, this object does some work when it is dequeued by the selecting thread and some work when the associated operation is ready to execute (after a select).

An initialize() method is not standard in a command object. In the NIO Manager library, the initialize() method typically enables the requested asynchronous operation by setting the channel interest operations, and the execute method() executes the operation, inform listeners about the performed operation, and sets the interest operations to a default value, previous value (before the initialize() method), or a value chosen by the application programmer through a parameter passed to the constructor of the object.

The initiate() method of the accept command configures the passed factory channel to be non blocking and registers the factory channel with the selector, specifying the initial interest operations as the Accept operation. When an accept operation executes, a new data channel is created for the client that sent the corresponding connection. The

execute() method of the accept command registers the new data channel with the selector object on which the selecting thread blocks. Like other command classes, the constructor of the accept command takes a nextInterestOp value. In this command, it determines the initial interest operations for this new data channel. The interest operations of the factory channel are not changed, so that additional client connections can be accepted.

The initiate() method of the connect command configures the individual channel to be non-blocking. A newly created channel is ready to connect() and is guaranteed a kernel buffer to send the connect message to the server. Thus, there is no need to wait for it to be ready to do this operation. So the initiate method() invokes the connect() method on the channel to send a message to the server. It then registers the channel with the selector object with the interest operation specifying the code for connect. The operation completes when an accept message is received from the server, which unblocks the selecting thread interested in this operation. At this point, the execute() method of the command object is called, which calls the finishConnect() operation of the channel to determine if the connection succeeded, and on success calls the registered application connect listener. If a nextInterestOp was passed to the constructor, it sets the channel interest operations to this value. Otherwise it sets them to 0. (Not changing the connect interest op will cause the next select() operation to unblock, surprisingly!)

The read queue of a channel consists of a single command object. Its initiate method is called whenever the application invokes the NIO Manager enableReads() method for the channel, which sets the next interest operations to Read. On each ready read, the execute() method of the command object is called. This object creates a single, large, read buffer, which is used to receive all incoming data for the channel. This buffer is passed directly to the listeners. If these contents must be accessed after the next read, they should be copied, as mentioned above. The execute() method of the command object changes the interest operations to those passed to the constructor of the command object, if this value is non-null. Otherwise, it makes no changes to them.

The selecting thread gives precedence to accept and connect queues over the write and read queues.

Once it starts processing the write queue, it disables all other operations until the write queue is empty. The byte buffer in a write command may not be emptied in one write, so it may be involved in multiple channel writes. The pointers in the byte buffer indicate how much of it is written. The write command is dequeued from its queue only when all of it is written. As mentioned above, all queued writes are attempted in one loop, before processing any other queued operation. A single initiate() method on a non-empty write-command queue (rather than a queue element) is invoked each time the selecting thread is woken up, which sets the interest operations to the write operation, saving the previous interest operations. The woken selecting thread then blocks to wait for operations of interest to be executable, which now is the write operation, and when the write operation on a write channel is ready, calls the execute() method of each queued write on that channel until the method returns false to indicate not all of the bytes in the write could actually be written. The write command is dequeued when all of its contents are written. When the byte buffer of a write command is not completely written, the interest operations are reset to write and the selecting thread blocks again on the select()

operation. When the write queue is empty, the interest operations are set to the saved values. The listener of an empty buffer notification can change these operations.

The selecting thread is independent of the implementation of the command objects and the queues in which they are put. It uses factories for creating them, which are retrieved from factory selector objects. Application programs can change the predefined factories stored in these selector objects.

The selectors are in the package inputport.nio.manager.factories.selectors and the predefined factories are in the package inputport.nio.manager.factories.classes.

The NIO manager provides a set of default factory objects, which will be sufficient for this assignment. You need to know only about the factory classes for connect and accept, which are referenced in the example, and have constructors that specify interestops passed to the constructors of the command objects they create

The NIO manager relieves a channel reader from the task of scattering gathered buffers and gathering scattered buffers. Before writing an application buffer, it sends a header buffer indicating the size of the application buffer. Based on this size, it performs the scatter-gather and gather-scatter tasks, invoking the read listener methods with buffers that are equal to those sent. The buffers passed to the read listener contains an initial component specifying the header- size of the sent bytes- and then a payload component containing the sent message.

The example print server and client illustrates the NIO Manager and NIO.

Enabling Server Accepts

The server, executed first, produces the following trace:

```
1. I***@2048388574049500 {Selecting Thread}(SelectCalled)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851


2. I***@2049359523166800 {main}(SocketChannelBound)
   EvtSrc(AnNIOManagerPrintServer)  0.0.0.0/0.0.0.0:9000
   sun.nio.ch.ServerSocketChannelImpl[/0:0:0:0:0:0:0:0:9000]

3. I***@2048388574051300 {main}(SelectorFactorySet)
   EvtSrc(AcceptCommandFactorySelector)
   inputport.nio.manager.factories.classes.AnAcceptCommandFactory@71bbf57e

4. I***@2048388575904800 {main}(ListenableAcceptsEnabled) EvtSrc(AnNIOManager)
   [examples.nio.manager.print.AnNIOManagerPrintServer@23faf8f2]<-sun.nio.ch.S
   erverSocketChannelImpl[/0:0:0:0:0:0:0:0:9000]

5. I***@2048388586582400 {main}(SelectorRequestEnqueued)
   EvtSrc(ASelectionConnectionManager)
   Command:inputport.nio.manager.commands.classes.AnAcceptCommand@56aac163
```

```
6. I***@2048388586959200 {main}(SelectorWokenUp)
   EvtSrc(ASelectionConnectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851


7. I***@2048388587539600 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851

8. I***@2048388588003000 {Selecting Thread}(SelectorRequestDequeued)
   EvtSrc(ASelectionConnectionManager)
   Command:inputport.nio.manager.commands.classes.AnAcceptCommand@56aac163

9. I***@2048388588457300 {Selecting Thread}(SocketChannelBlockingConfigured)
   EvtSrc(AnAcceptCommand)  false
   sun.nio.ch.ServerSocketChannelImpl[/0:0:0:0:0:0:0:0:9000]

10. I***@2048388589255600 {Selecting Thread}(SocketChannelRegistered)
    EvtSrc(AnAcceptCommand)  Ops:OP_ACCEPT,
    Selector:sun.nio.ch.WindowsSelectorImpl@404ab851,
    sun.nio.ch.ServerSocketChannelImpl[/0:0:0:0:0:0:0:0:9000]

11. I***@2048388589550600 {Selecting Thread}(SelectCalled)
    EvtSrc(AScatterGatherSelectionManager)
    sun.nio.ch.WindowsSelectorImpl@404ab851
```

Here is what the traced steps do:

1. Make the selecting thread execute select to block for ready operations.
2. Bind the logical endpoint to the int port number, 9000.
3. Ask the NIO manager to use a factory for creating accept command objects that passes to the command object the Read operation (code 1) as the initial interest operation for the new individual channel accepted by the command. This step simply sets the factory. The factory is used to create the accept command in step 4.
4. Ask the NIOManager to accept incoming connect messages for the specified channel factory, and register a listener (`examples.nio.manager.print.AnNIOManagerPrintServer@23faf8f2`) to be invoked when the accept request finishes execution (create the new data channel).
5. Enqueue the accept request for the selecting thread.
6. Wake up the selecting thread so that it can process the request.
7. The selecting thread unblocks. The remaining steps are executed by the selecting thread rather than the main thread.
8. Dequeue the accepted request.
9. Configure the channel endpoint to be non-blocking.
10. Register the factory endpoint with the selector with the interest op ACCEPT_OP (16 is the code for accept). Thus, ask the selector to wait for the accept command to be executable on the factory channel, not to be confused with the read op to be associated with the data channel created when the accept completes.

11. Call select() again to wait for the end-point channel to be executable or to be woken up again by an application thread.

The accept command will not be executable until some client connects to the end-point.

Processing a Connect in the Client

Let us run a client that connects:

```
1. I***@2051062806716700 {Selecting Thread}(SelectCalled)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
2. I***@2051062806717300 {main}(SelectorFactorySet)
   EvtSrc(ConnectCommandFactorySelector)
   inputport.nio.manager.factories.classes.AConnectCommandFactory@631330c
3. I***@2051062810181400 {main}(SocketChannelConnectRequested)
   EvtSrc(AnNIOManager)
   (localhost/127.0.0.1:9000,[examples.nio.manager.print.AnNIOManagerPrintC
   lient@27808f31]) <-java.nio.channels.SocketChannel[unconnected]
4. I***@2051062814213000 {main}(SelectorRequestEnqueued)
   EvtSrc(ASelectionConnectionManager)
   Command:inputport.nio.manager.commands.classes.AConnectCommand@2c039ac6
5. I***@2051062814545300 {main}(SelectorWokenUp)
   EvtSrc(ASelectionConnectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
6. Enter a line of input to be printed remotely
7. I***@2051062815182500 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
8. I***@2051062815688600 {Selecting Thread}(SelectorRequestDequeued)
   EvtSrc(ASelectionConnectionManager)
   Command:inputport.nio.manager.commands.classes.AConnectCommand@2c039ac6
9. I***@2051062816076300 {Selecting
   Thread}(SocketChannelBlockingConfigured) EvtSrc(AConnectCommand)  false
   java.nio.channels.SocketChannel[unconnected]
10. I***@2051062816937200 {Selecting Thread}(SocketChannelConnectInitiated)
    EvtSrc(AConnectCommand)
    java.nio.channels.SocketChannel[connection-pending
    remote=localhost/127.0.0.1:9000]
11. I***@2051062817856200 {Selecting Thread}(SocketChannelRegistered)
    EvtSrc(AConnectCommand)  Ops:OP_CONNECT,
    Selector:sun.nio.ch.WindowsSelectorImpl@580af1d8,
    java.nio.channels.SocketChannel[connection-pending
    remote=localhost/127.0.0.1:9000]
12. I***@2051062818125300 {Selecting Thread}(SelectCalled)
    EvtSrc(AScatterGatherSelectionManager)
    sun.nio.ch.WindowsSelectorImpl@580af1d8
13. I***@2051062818412500 {Selecting Thread}(SelectUnblocked)
    EvtSrc(AScatterGatherSelectionManager)
    sun.nio.ch.WindowsSelectorImpl@580af1d8
14. I***@2051062819401400 {Selecting Thread}(SocketChannelConnected)
    EvtSrc(AConnectCommand)
    [examples.nio.manager.print.AnNIOManagerPrintClient@27808f31]<-java.nio.
    channels.SocketChannel[connected local=/127.0.0.1:51043
    remote=localhost/127.0.0.1:9000]
```

```
15. I***@2051062819746100 {Selecting Thread}(SocketChannelInterestOp)
    EvtSrc(AConnectCommand)  Ops:NO_OP Selection
    Key:sun.nio.ch.SelectionKeyImpl@5bc99728
16. I***@2051062819973900 {Selecting Thread}(SelectCalled)
    EvtSrc(AScatterGatherSelectionManager)
    sun.nio.ch.WindowsSelectorImpl@580af1d8
```

It creates an unconnected individual channel and asks the NIOManager to connect to the serverchannel endpoint (named by the host and port), specifying the connect listener, `examples.nio.manager.client.AMeaningOfLifeNIOClient@26aa12dd` , to be called when the connect succeeds.

A connect command is created from this request and enqueued in the connect-command queue. When this command is dequeued by the selection manager, the initiate() method of the command is executed. This method configures the individual channel to be non-blocking. It registers the channel with the selector object with interest operations specifying the code for connect (code, 8). It then invokes the connect() method on the channel to send a connect message to the server. The server responds with an accept message, which unblocks the selecting client thread interested in this operation. The execute() method of the connect command is called, which invokes the registered application connect listener after calling finishConnect() on the channel. It sets the interest operations to 0 (printed as NO_OP) – as this was the interest op was passed to the constructor of the factory that created this command.

About the same time, the main thread prompts the user for input.

Processing an Accept in the Server

Let us return to the server to see how the connect message from the client was processed:

```
1. I***@2051724433346100 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851
2. I***@2051724434143900 {Selecting Thread}(SocketChannelAccepted)
   EvtSrc(AnAcceptCommand)
   [examples.nio.manager.print.AnNIOManagerPrintServer@2d6eabae]<-java.nio.
   channels.SocketChannel[connected local=/127.0.0.1:9000
   remote=/127.0.0.1:51067]:sun.nio.ch.ServerSocketChannelImpl[/0:0:0:0:0:0
   :0:0:9000]
3. I***@2051724434517600 {Selecting Thread}(SocketChannelRegistered)
   EvtSrc(AnAcceptCommand)  Ops:OP_READ,
   Selector:sun.nio.ch.WindowsSelectorImpl@404ab851,
   java.nio.channels.SocketChannel[connected local=/127.0.0.1:9000
   remote=/127.0.0.1:51067]
4. I***@2051724434811200 {Selecting Thread}(ReadListenerAdded)
   EvtSrc(AnNIOManager)
   examples.nio.manager.print.AnNIOManagerPrintServer@2d6eabae<-java.nio.ch
   annels.SocketChannel[connected local=/127.0.0.1:9000
   remote=/127.0.0.1:51067]
5. I***@2051724442042300 {Selecting Thread}(SelectCalled)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851
```

The incoming message unblocks the selecting thread. The thread finds the associated accept command (based on the end-point specified by the connect message), and invokes the execute() method of the command object. This method invoked the ready accept

operation on the channel endpoint, which returns a new individual data channel connected to the client channel. It registers the channel with the selector, expressing interest in the channel's READ_OP – the operation passed to its constructor. It calls the listener method (in the registered listener, `examples.nio.manager.print.AnNIOManagerPrintServer@2d6e`), passing it the newly created individual socket channel. This application method asks the NIO Manager to register a read listener (`examples.nio.manager.print.AnNIOManagerPrintServer@2d6eabae`) for the individual channel. The selector then blocks on the selector's select();

Processing a (Client) Write

Here is what happens in the client when the user enters the value "The wood are lovely dark and deep":

```
The wood are lovely dark and deep
1. I***@2051752823072600 {main}(SocketChannelWriteRequested)
   EvtSrc(AnNIOManager)  java.nio.HeapByteBuffer[pos=0 lim=33 cap=33],
   [examples.nio.manager.print.AnNIOManagerPrintClient@27808f31]
   java.nio.channels.SocketChannel[connected local=/127.0.0.1:51067
   remote=localhost/127.0.0.1:9000]
2. I***@2051752897120200 {main}(WriteRequestEnqueued)
   EvtSrc(AWriteBoundedBuffer)
   Command:inputport.nio.manager.commands.classes.AHeaderWriteCommand@4abdb505
3. I***@2051752897309600 {main}(WriteRequestEnqueued)
   EvtSrc(AWriteBoundedBuffer)
   Command:inputport.nio.manager.commands.classes.AWriteCommand@23d2a7e8
4. I***@2051752897451600 {main}(SelectorWokenUp)
   EvtSrc(AScatterGatherSelectionWriteManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
5. Enter a line of input to be printed remotely
6. I***@2051752897727500 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
7. I***@2051752898031800 {Selecting Thread}(SocketChannelInterestOp)
   EvtSrc(AWriteBoundedBuffer)  Ops:OP_WRITE Selection
   Key:sun.nio.ch.SelectionKeyImpl@5bc99728
8. I***@2051752898161900 {Selecting Thread}(SelectCalled)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
9. I***@2051752898303200 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@580af1d8
10. I***@2051752899459000 {Selecting Thread}(SocketChannelWritten)
   EvtSrc(AHeaderWriteCommand)  java.nio.HeapByteBuffer[pos=0 lim=4 cap=4]
   java.nio.channels.SocketChannel[connected local=/127.0.0.1:51067
   remote=localhost/127.0.0.1:9000]
11. I***@2051752899882100 {Selecting Thread}(WriteRequestDequeued)
   EvtSrc(AWriteBoundedBuffer)
   Command:inputport.nio.manager.commands.classes.AHeaderWriteCommand@4abdb505
12. I***@2051752900020200 {Selecting Thread}(SocketChannelWritten)
   EvtSrc(AWriteCommand)  java.nio.HeapByteBuffer[pos=0 lim=33 cap=33]
   java.nio.channels.SocketChannel[connected local=/127.0.0.1:51067
   remote=localhost/127.0.0.1:9000]
```

```
13. I***@2051752900276300 {Selecting Thread}(WriteRequestDequeued)
    EvtSrc(AWriteBoundedBuffer)
    Command:inputport.nio.manager.commands.classes.AWriteCommand@23d2a7e8
14. I***@2051752900413000 {Selecting Thread}(SocketChannelInterestOp)
    EvtSrc(AWriteBoundedBuffer)  Ops:NO_OP Selection
    Key:sun.nio.ch.SelectionKeyImpl@5bc99728
15. I***@2051752900587800 {Selecting Thread}(WriteBufferIsEmpty)
    EvtSrc(AWriteBoundedBuffer)  []<--java.nio.channels.SocketChannel[connected
    local=/127.0.0.1:51067 remote=localhost/127.0.0.1:9000]
16. I***@2051752900752500 {Selecting Thread}(SelectCalled)
    EvtSrc(AScatterGatherSelectionManager)
    sun.nio.ch.WindowsSelectorImpl@580af1d8
```

The main thread converts the input string to a byte buffer and makes a write request to the NIO Manager. This results in two write commands being enqueued in a write command queue, a header command that writes the size of the application byte buffer, and another that writes the application byte buffer. Next the selecting thread is woken up.

The woken thread finds a non-empty write command queue, sets the interestops to WRITE_OP (code 4), and waits for this operation to be ready. When unblocked, it calls the execute method of each of the two pending write operations, which complete successfully and dequeue themselves. It also calls any pending write listeners, which can use the buffer for other purposes). When the write command queue is empty, the interest operations are set to NO_OP (0), the saved and initial value for the channel), and listeners waiting for an empty buffer are called. In this case, there is no such listener.

Processing a (Server) Read

The following reading steps occur in the server in response to a client write being communicated to the server:

```
1. I***@2051752899879700 {Selecting Thread}(SelectUnblocked)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851
2. I***@2051752902961400 {Selecting Thread}(SocketChannelRead)
   EvtSrc(AScatterGatherReadCommand)  (37)java.nio.HeapByteBuffer[pos=37
   lim=4194304 cap=4194304] java.nio.channels.SocketChannel[connected
   local=/127.0.0.1:9000 remote=/127.0.0.1:51067]
3. I***@2051752903310900 {Selecting Thread}(SocketChannelHeaderRead)
   EvtSrc(AScatterGatherReadCommand)  (33)java.nio.HeapByteBuffer[pos=4 lim=37
   cap=4194304] java.nio.channels.SocketChannel[connected
   local=/127.0.0.1:9000 remote=/127.0.0.1:51067]
4. I***@2051752903630300 {Selecting Thread}(SocketChannelFullMessageRead)
   EvtSrc(AScatterGatherReadCommand)
   [examples.nio.manager.print.AnNIOManagerPrintServer@2d6eabae]<-(33)java.nio
   .HeapByteBuffer[pos=4 lim=37 cap=4194304]
   java.nio.channels.SocketChannel[connected local=/127.0.0.1:9000
   remote=/127.0.0.1:51067]
5. The wood are lovely dark and
   deep<--java.nio.channels.SocketChannel[connected local=/127.0.0.1:9000
   remote=/127.0.0.1:51067]
```

```
6. I***@2051752903934500 {Selecting Thread}(SelectCalled)
   EvtSrc(AScatterGatherSelectionManager)
   sun.nio.ch.WindowsSelectorImpl@404ab851
```

Both of the client writes are gathered into a single message read by the server. The NIO manager reads the first 4 bytes of the message to determine that the application message is 33 bytes long. It then creates a new byte buffer that shares the array of the read byte buffer and encapsulates the next 6 bytes (through the buffer pointers) and sends it to the read listeners. The sole read listener converts the byte buffer to a string and prints it along with the channel on which it was received.